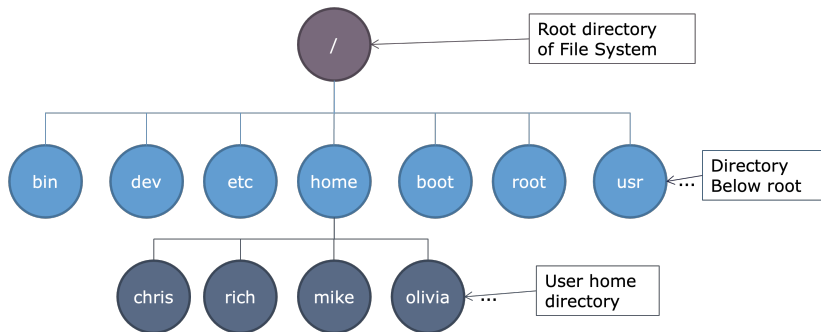


Linux Hierarchical Directory

Linux Hierarchical Directory

For a Linux developer, it is important to know where tools and development resources are located.



Standard Directories for System Programs

- `/bin` stores binaries, programs used in booting the system.
- `/dev` stores device files such as device drivers for I/O devices.
- `/etc` stores all system related configuration (.conf) files.
- `/home` stores all user's home directory.
- `/boot` stores everything required for the boot process (kernel file, boot loader...)
- `/root` home directory of the System Administrator.
- `/usr` stores the largest share of data on a system (all the user binaries, their documentation, libraries, header files, etc)
- `/lib` stores kernel modules and those shared library images needed to boot the system and run the commands in the root file system.

Standard Directories for Application Programs

Where to locate user application programs such as `gcc` compiler?

- `/usr/bin` stores applications supported by system for general user.
- `/usr/local/bin` or `/usr/local/opt` stores applications for specific host computer or local networks, added by system admin.
- `gcc` compilers are located in `/usr/bin` or `/usr/local/bin`.
- Additional features and programming system such as X Window is commonly installed in the `/usr/X11` or `/usr/bin/X11` directory.

Locating Header Files

- For programming in C and other languages, you need header files to provide definitions of constants and declarations for system and library function calls.
- For C, the header files are almost always located in `/usr/include` and subdirectories thereof. For example,
 - `/usr/include/sys` stores header files for system
 - `/usr/include/linux` stores header files for Linux
 - `/usr/include/X11` stores header files for X Window
- You can use header files in subdirectories or nonstandard places by specifying the `-I` flag (for include) to the C compiler. For example,
`gcc -I/usr/openwin/include fred.c`
will direct the compiler to look in the directory `/usr/openwin/include`, as well as the standard places, for header files included in the `fred.c` program.

Locating Library Files

- Libraries are collection of precompiled functions.
- Standard library locations:
 - `/lib`
 - `/usr/lib`
- A library filename always starts with `lib`. Then follows the part indicating what library this is (like `c` for the C library, or `m` for the mathematical library). The last part of the name starts with a dot (`.`), and specifies the type of the library:
 - `.a` for traditional, static libraries
 - `.so` for shared libraries

Locating Library Files (Cont')

- Linker knows the standard libraries locations.
- Users can also instruct the linker to search a library at a specific location by passing the name of library with full path. For example,
`gcc -o fred fred.c /usr/lib/libm.a`
or use shorthand command:
`gcc -o fred fred.c -lm`
where `-lm` denotes `libm.a` located in one of the standard directories.
- Although libraries are usually found in standard places, you can add to the search directories by using the `-L` (uppercase letter) flag to the compiler. For example,
`gcc -o x11fred -L/usr/openwin/lib x11fred.c -lX11`
will compile and link a program called `x11fred` using the version of the library `libX11` found in the `/usr/openwin/lib` directory.

Static Libraries (aka. Archives)

- Static Libraries is a collection of object files kept together in a ready-to-use form.
- To use a function in the library, just include a header file in your program.
- You can create and maintain your own static libraries very easily by using the `ar` (for archive) program and compiling functions separately with `gcc -c`.

Steps to Create Your Own Static Libraries

- 1 Create separate source files (`fred.c` and `bill.c`) for each function.
- 2 Create a header file (`mylib.h`) which contains function prototypes.
- 3 create a test file `program.c` that utilize/include the `mylib.h`.
- 4 You can now compile and test the program without the library.

```
gcc -c program.c fred.c bill.c
gcc -o program program.o bill.o fred.o
./program
```

- 5 Now, to create a library, use `ar` program:

```
ar crv libfoo.a bill.o fred.o
```

- 6 Your library is now ready to use.

```
gcc -o program program.o libfoo.a
./program
```

- 7 You could also use the `-l` option to access the library, but because it is not in any of the standard places, you have to tell the compiler where to find it by using the `-L` option like this:

```
gcc -o program program.o -L. -lfoo
```

Code Examples

```
/* bill.c */
#include <stdio.h>
void bill (char *arg){
    printf("Bill: You passed %s\n ", arg);
}

/*fred.c */
#include <stdio.h>
void fred (int arg){
    printf("Fred! You are passed: %d\n", arg);
}

/* A header file mylib.h containing the two function prototype
*/
void bill (char *);
void fred (int);

/* program.c which include main function*/
#include <stdlib.h>
#include "mylib.h"
int main () {
    bill(" Hello world ");
    fred (100);
    exit(0);
}
```

Disadvantage of Static Libraries

- One disadvantage of static libraries is that when you run many applications at the same time and they all use functions from the same library, you may end up with many copies of the same functions in memory and indeed many copies in the program files themselves.
- Many UNIX systems and Linux-support shared libraries can overcome this disadvantage.

Shared Libraries

- Shared libraries are stored in the same places as static libraries, but with `.so` filename suffix. On a typical Linux system, the shared version of the standard math library is `/lib/libm.so`
- When a program uses a shared library, it is linked in such a way that it doesn't contain function code itself, but references to shared code that will be made available at run time.
- In many ways, shared libraries are similar to dynamic-link libraries used under Windows. The `.so` libraries correspond to `.DLL` files and are required at run time.
- A shared library can be created by any user with sufficient permissions to write to the file system.
- However, installing the shared library in a system-wide location, such as `/usr/lib` or `/usr/local/lib`, typically requires root privileges, so that system-wide shared libraries can be shared by all users on the system.

How to Create and Use Shared Libraries

Assume we have the following 3 programs:

- `shared.c` defines functions to be shared.
- `shared.h` declares function prototypes.
- `example.c` program to test shared library.

Code Example

```
//example.c
#include<stdio.h>
#include"shared.h"
int main(void) {
    unsigned int a = 7;
    unsigned int b = 4;
    unsigned int add_result, sub_result, mult_result;

    add_result = add(a,b);
    sub_result = subtract(a, b);
    mult_result = mult(a, b);

    printf("\n %u + %u = %u \n",a, b, add_result);
    printf("\n %u - %u = %u \n",a, b, sub_result);
    printf("\n %u * %u = %u \n",a, b, mult_result);
    return 0;
}
```

Code Example (Cont')

```
// shared.h
#include <stdio.h>
extern unsigned int add(unsigned int a, unsigned int b);
extern unsigned int subtract(unsigned int a, unsigned int b);
extern unsigned int mult(unsigned int a, unsigned int b);
```

```
// shared.c
#include "shared.h"
unsigned int add(unsigned int a, unsigned int b) {
    return (a+b);
}
unsigned int subtract(unsigned int a, unsigned int b){
    return (a-b);
}
unsigned int mult(unsigned int a, unsigned int b){
    return (a*b);
}
```

Commands to Create and Use a Shared Library

- To compile the source code to **Position Independent Code (PIC)**.

```
gcc -c -fPIC shared.c
```

- To create a shared library `libshared.so`

```
gcc -shared -o libshared.so shared.o
```

- To test the shared library:

```
gcc -c example.c
```

```
gcc -o example example.o libshared.so
```

```
./example
```

- As like static library, use `-L` flag to specify the nonstandard location of the shared libraries. For example:

```
gcc -L. example.c -o example -lshared
```

```
./example
```